Deploying a Full Wazuh SIEM Architecture

End-to-End Build, Detection Engineering & Real-World Attack Testing

Environment: Ubuntu 26.04 • Windows 11 Enterprise • Wazuh v4.14.5 | **Author:** Ihar Shcharbitski

1. Lab Architecture

Three machines, three distinct roles — mirroring how real organizations separate the system being watched from the system doing the watching.

Role	Machine	Function
Wazuh Manager (Server)	Ubuntu 26.04 VM	Central analysis engine. Receives events from agents, decodes them, runs the rule engine, assigns severity, and generates alerts.
Wazuh Indexer	Ubuntu 26.04 VM	OpenSearch-based datastore. Stores every processed alert for search and visualization.
Wazuh Dashboard	Ubuntu 26.04 VM	HTTPS web UI. Queries the Indexer in real time and renders charts, tables, and searches.
Wazuh Agent	Windows 11 VM	Lightweight endpoint collector. Monitors Windows Event Log + Sysmon; ships raw telemetry to the Manager.

> **Architecture Note**
> "Wazuh server" and "Wazuh manager" are the same component — the manager daemon is the analysis brain of the server. In this lab, the Manager, Indexer, and Dashboard all run on the same Ubuntu VM (an "all-in-one" deployment). In production environments, each component runs on a dedicated host for scalability and redundancy.

2. Architecture & Conceptual Foundation

Before proceeding to hands-on steps, it is essential to understand the internal data flow of a SIEM. A surface-level understanding leads to blind configuration. The following section establishes the conceptual foundation underlying every practical step in this lab.

2.1 Component Deep-Dive

Component	Definition	Key Functions	Communication & Ports
Wazuh Agent	Lightweight endpoint monitoring service.	Collects Windows Event Logs, Sysmon events, and file integrity data. Ships raw data outward with minimal local analysis.	Ports 1514 (ongoing data) and 1515 (enrollment handshake) → Manager.
Wazuh Manager	Central analysis engine and server daemon.	Decodes raw events into structured fields. Evaluates events against the rule engine. Assigns severity levels and generates alerts.	Inbound: 1514/1515 from Agents. Outbound: 9200 to Indexer via Filebeat.
Wazuh Indexer	Searchable datastore built on OpenSearch.	Indexes and stores all Manager-generated alerts. Provides rapid search, filtering, and aggregation.	Receives from Manager via 9200; serves queries to Dashboard.
Wazuh Dashboard	Web-based UI built on OpenSearch Dashboards.	Renders real-time visual charts, tables, and search interfaces. No local data storage or analysis.	Connects exclusively to the Indexer on 9200.

> **Data Flow**
> Data moves strictly in a unidirectional pipeline: Agent → Manager → Indexer → Dashboard Agent collects • Manager thinks • Indexer remembers • Dashboard displays

2.2 The Wazuh Agent Is System-Wide, Not Per-User

The Wazuh agent runs as a Windows service (not a per-user application), started at boot under the SYSTEM account. This means it is fully independent of who is logged in, capturing all logon attempts, process creations, and network connections system-wide — across every user account on the endpoint, including the test accounts used during attack simulation.

2.3 Severity Levels

Every Wazuh rule carries a numeric level from 0 to 15. This single number determines triage priority in the Dashboard.

Level	Band	Meaning	Example Trigger
0	Ignored	No security relevance. Suppresses known-noisy, harmless events.	Routine cron job success
1–3	Low	Informational. Minor anomalies with no standalone relevance.	Unknown user generated a benign log entry
4–6	Medium-Low	Worth noting; typically correlated with other events.	Firewall rule changed; update installed

Level	Band	Meaning	Example Trigger
7–8	Medium	Suspicious activity deserving analyst review.	Multiple failed logon attempts (early-stage)
9–11	High	Strong indicator of compromise or active attack technique.	Successful brute-force; malware signature match
12–13	Very High	Multiple correlated high-severity events.	Privilege escalation + new admin account
14–15	Critical	Near-certain active compromise. Immediate response required.	Audit log cleared (T1070.001)

2.4 The Rule Engine

Every Wazuh rule is defined in XML.

The fundamental distinction: single-event rules look at one log line and decide whether it matches. Correlation rules track how often other rules have already fired, over a time window. Everything else in the rule syntax serves one of these two mechanisms.

Attribute	Meaning
id	Unique rule identifier. Built-in rules: below 100000. Custom rules: 100000–120000 (avoids collision with official ruleset).
level	Severity assigned when the rule fires (0–15).
frequency	Number of matches required before this correlation rule fires.
timeframe	Rolling window in seconds within which the frequency threshold must be reached.
if_matched_sid	Correlation: fires when the specified rule has fired N times within a time window. Counts rule firings, not log lines.
if_sid	Single-event: fires only if the specified parent rule already matched on this same log line. Builds parent–child chains.
if_matched_group	Correlation: same as if_matched_sid but counts any rule sharing a given group tag — useful for multi-rule correlation.
field / win.eventdata.X	Matches against a specific decoded field (IP, username, command line) rather than raw log text.
same_field / different_field	Requires that correlated events share the same field value (brute force from one IP) or differ (password spray across accounts).
options: no_full_log	Suppresses storing the full raw log in alerts.json, reducing storage. Applied once rule logic is trusted.
group (child element)	Adds category tags to this specific rule for Dashboard filtering and compliance mapping (PCI-DSS, GDPR, HIPAA).
mitre / id	Attaches a MITRE ATT&CK technique ID directly. Populates the Dashboard MITRE module automatically.

2.5 SIEM Market Context

Wazuh is not the only SIEM in production environments. Understanding the landscape adds context to tooling decisions:

- Wazuh — Open source, agent-based, built-in detection rules, MITRE ATT&CK mapping, and compliance dashboards out of the box. Ideal for resource-constrained environments and self-hosted deployments.
- Elastic Stack (Elasticsearch + Logstash + Kibana) — Maximum flexibility with no built-in detection rules; analysts/engineers build and own everything. Open-core model; advanced security features are commercial.
- Splunk — Industry standard with the broadest enterprise ecosystem and integrations. Cutting-edge search language (SPL), extensive app marketplace, and strong vendor support — but carries significant licensing cost.

3. Installing the Wazuh Central Components

Wazuh provides an official installation assistant script that deploys all three central components (Manager, Indexer, Dashboard) on a single host in one pass. This is the supported quickstart method for lab and small-scale production deployments.

3.1 Preparation

- Ensure the VM has at least 4 GB RAM / 2 vCPU and 30+ GB of free disk space. The Indexer (OpenSearch) is the most memory-hungry component and will silently fail to start if under-provisioned (I went through that).
- Create a dedicated directory to keep the installer and its generated credential archive together:

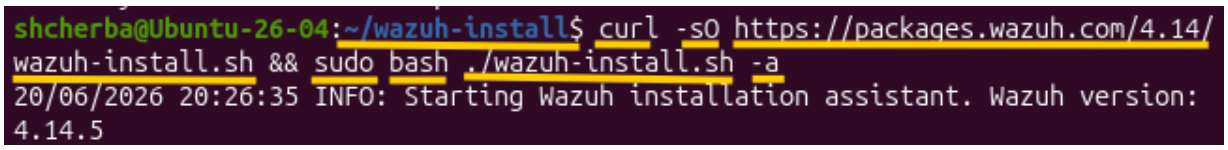
```
mkdir -p ~/wazuh-install
cd ~/wazuh-install
```

3.2 Download and Run the Installation Assistant

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh \
  && sudo bash ./wazuh-install.sh -a
```

- curl -sO — downloads the file silently (-s) and saves it under its original remote filename (-O) rather than printing to the terminal.
- -a — "all-in-one" flag. Installs Manager, Indexer, and Dashboard together on this single host. Without it, each component must be installed and wired up individually.

Behind the scenes the script: installs all component packages, auto-generates strong random passwords for every internal Wazuh user, generates self-signed TLS certificates for secure inter-component communication, and starts services.



Initiating the Wazuh installation assistant (wazuh-install.sh -a)

3.3 Retrieve Admin Credentials

At the end of a successful run, the script prints the Dashboard URL and the generated admin password. If the output was missed or the terminal was closed, recover credentials at any time with:

```
sudo tar -O -xvf ~/wazuh-install/wazuh-install-files.tar \
  wazuh-install-files/wazuh-passwords.txt
```

```
21/06/2026 01:22:37 INFO: --- Summary ---
21/06/2026 01:22:37 INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
    User:
    Password:
21/06/2026 01:22:37 INFO: Installation finished.
```

Installation summary — Dashboard URL and redacted admin credentials

> **Security Practice**
> NEVER paste the raw admin password into documentation, screenshots, or version-controlled files. Redact it before sharing.

3.4 Verify All Three Services Are Running

```
shcherba@Ubuntu-26-04:~/wazuh-install$ sudo systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
     Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-06-21 01:22:16 +03; 7min ago
 Invocation: 4a1a23b6f49647cb8c656ed013d60a7b
      Tasks: 211 (limit: 7477)
     Memory: 1.2G (peak: 1.2G)
        CPU: 56.303s
     CGroup: /system.slice/wazuh-manager.service
```

wazuh-manager: active (running) — Wazuh analysis engine confirmed healthy

```
shcherba@Ubuntu-26-04:~/wazuh-install$ sudo systemctl status wazuh-dashboard
● wazuh-dashboard.service - wazuh-dashboard
     Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-06-21 01:22:19 +03; 9min ago
 Invocation: e2b15840016f479cb60767f4370ccf45
   Main PID: 67131 (node)
      Tasks: 11 (limit: 7477)
     Memory: 190.6M (peak: 240.7M)
        CPU: 9.331s
     CGroup: /system.slice/wazuh-dashboard.service
             └─67131 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536
```

wazuh-dashboard: active (running) — web UI service confirmed

```
shcherba@Ubuntu-26-04:~/wazuh-install$ sudo systemctl status wazuh-indexer
● wazuh-indexer.service - wazuh-indexer
     Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-06-21 01:08:46 +03; 24min ago
 Invocation: 5218160014bd43e5baba42e398587cf4
       Docs: https://documentation.wazuh.com
   Main PID: 11856 (java)
      Tasks: 72 (limit: 7477)
     Memory: 1.3G (peak: 1.3G)
        CPU: 1min 6.878s
     CGroup: /system.slice/wazuh-indexer.service
             └─11856 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress
```

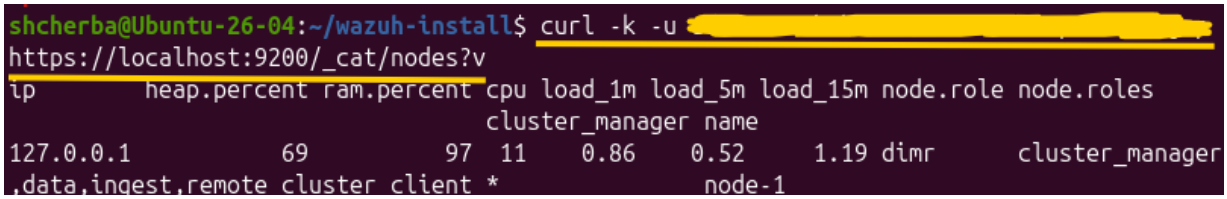
wazuh-indexer: active (running) — OpenSearch cluster confirmed

3.5 Verify Indexer Cluster Health via API

Checking systemctl status only confirms the process is alive. Querying the Indexer's REST API confirms it has completed its full internal startup sequence (JVM initialization, security plugin, TLS, and cluster formation) and is actually accepting requests:

```
curl -k -u <credentials> https://localhost:9200/_cat/nodes?v
```

- -k — skips TLS certificate validation. Required because the lab uses a self-signed certificate that curl would otherwise reject.
- /_cat/nodes?v — OpenSearch endpoint returning a tabular node list. ?v adds column headers. A healthy single-node lab returns exactly one row.



Single-node OpenSearch cluster confirmed healthy — all roles on node-1

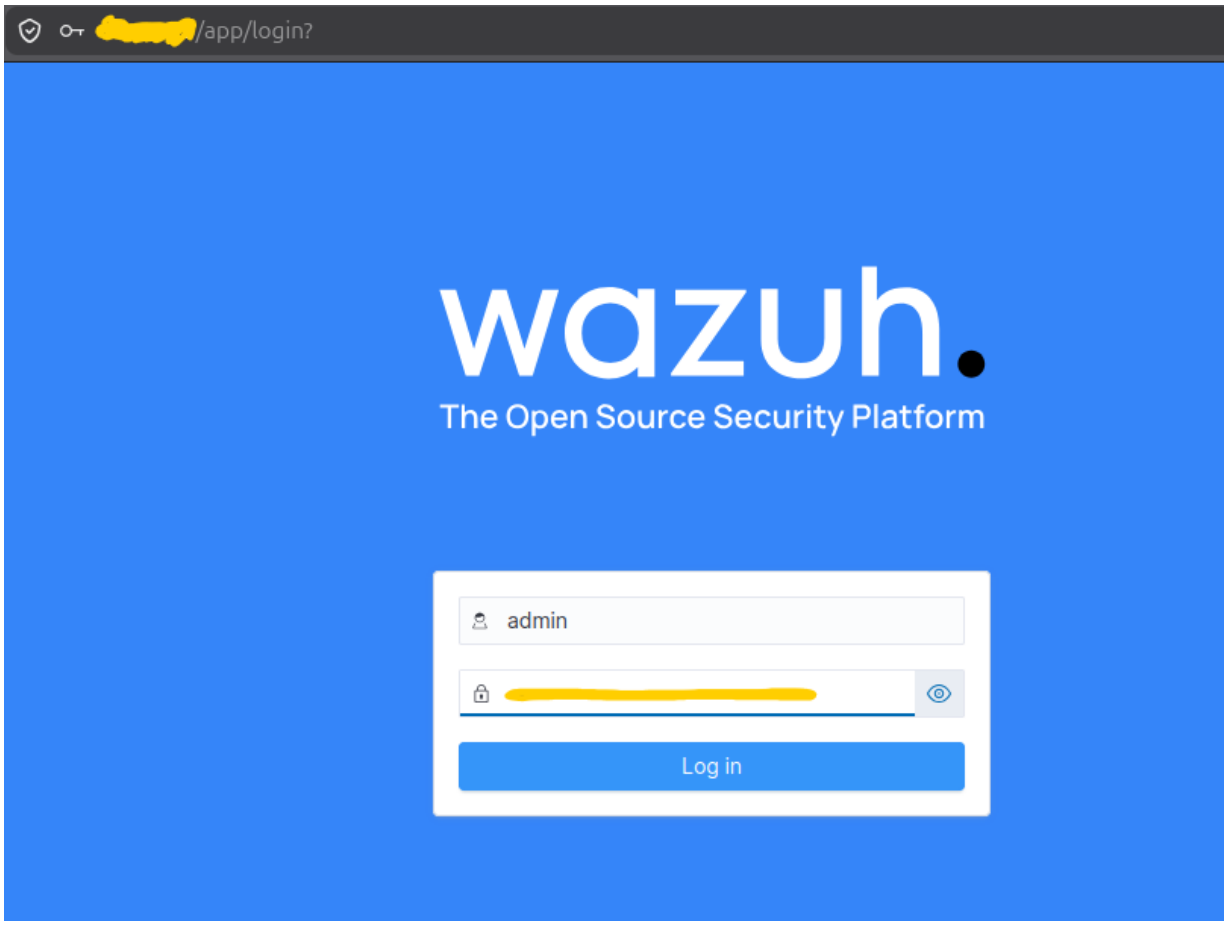
> **Cluster Concept**
> A cluster is a group of one or more servers ("nodes") working together as a single logical database. Even a 1-node setup is technically "a cluster with one member" — the same OpenSearch engine runs regardless of node count, which is why cluster terminology appears throughout the output.

3.6 Access the Dashboard

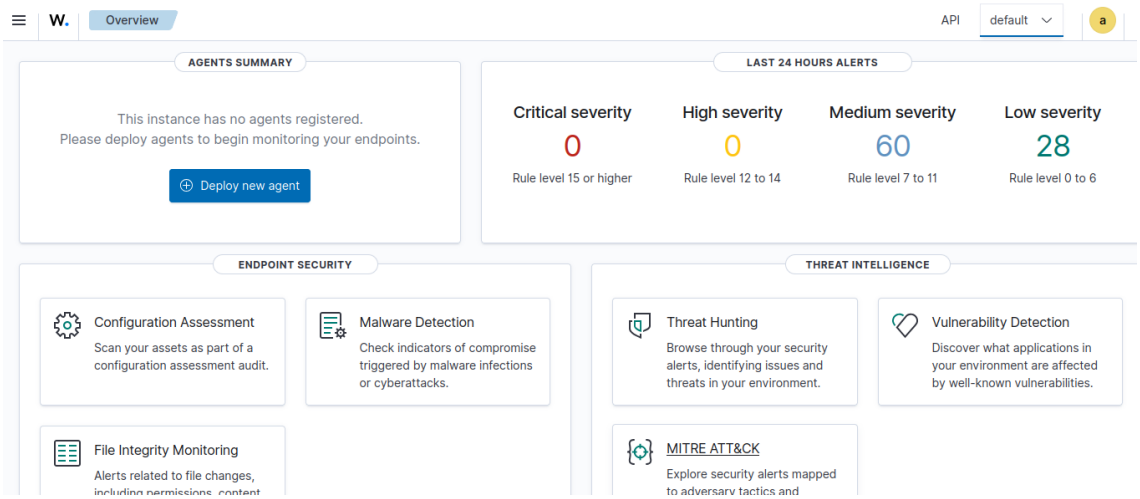
From a browser on the host machine (via VirtualBox port-forwarding) or from inside the Windows VM over the internal network:

```
https://<ubuntu-vm-ip>
```

Accept the browser's self-signed certificate warning. Log in with the admin username and the password retrieved in Section 3.3.



Wazuh Dashboard login page — credentials redacted



Dashboard Overview — Agent 000 (self-monitoring Ubuntu host) visible with live alerts

Agent 000 — Self-Monitoring

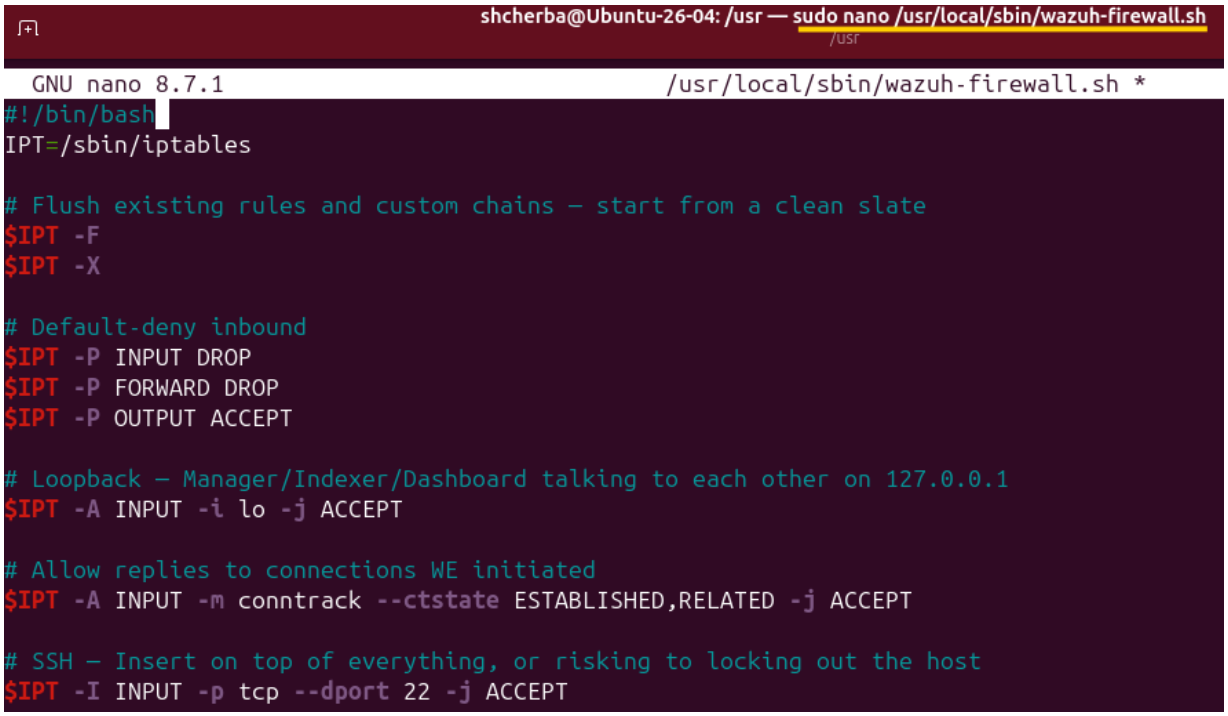
Wazuh automatically registers its own host server as Agent 000 and begins monitoring it immediately. The Manager hooks into the Ubuntu OS's local logs, authentication events, and health metrics without any additional configuration. This is expected and not an error.

4. Hardening & Post-Install Best Practices

4.1 Firewall Configuration with iptables

Rather than using the ufw wrapper, the firewall is implemented directly in iptables using an explicit, commented script. This approach is more representative of production systems and is portable across Linux distributions.

The script is saved at /usr/local/sbin/wazuh-firewall.sh. The /usr/local/sbin/ path is the conventional location for admin-authored system scripts: it is not managed by the package manager, never overwritten by apt, and by convention is only in root's $PATH — signalling that elevated privileges are expected. The actual privilege enforcement comes from the iptables commands themselves, which the kernel rejects if the calling process is not root.



Firewall script — whitelist model: default-deny inbound, loopback and stateful tracking accepted first

Wazuh-specific port rules — 1514/1515 (agent), 55000 (API), 443 (Dashboard)

Key iptables flags used:

Flag	Meaning
-F	Flush all existing rules from the named chain — starts from a clean slate rather than stacking on unknown prior state.
-X	Delete any custom (non-default) chains.
-P INPUT DROP	Sets the default policy to DROP — the whitelist model: nothing gets in unless explicitly permitted.
-A	Append a rule to the end of the named chain.
-I	Insert a rule at the top of the chain (used for SSH to ensure it is evaluated before any other rule).
-i lo	Matches traffic arriving on the loopback interface (127.0.0.1) specifically.
-m conntrack --ctstate ESTABLISHED,RELATED	Matches packets belonging to connections already initiated by this host, allowing replies without opening ports bidirectionally.
-p tcp --dport X	Matches TCP packets destined for port X.
-j ACCEPT	Accepts the matched packet.

4.2 Apply, Verify, and Persist Rules

```
shcherba@Ubuntu-26-04:/usr$ sudo chmod +x /usr/local/sbin/wazuh-firewall.sh
shcherba@Ubuntu-26-04:/usr$ sudo bash /usr/local/sbin/wazuh-firewall.sh
shcherba@Ubuntu-26-04:/usr$ sudo iptables -L -n -v --line-numbers
Chain INPUT (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:22
2      332 28460 ACCEPT     all  --  lo     *       0.0.0.0/0            0.0.0.0/0
3        0     0 ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0           ctstate RELATED,ESTABLISHED
4        0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:1515
5        0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:1514
6        0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:55000
7        0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0           tcp dpt:443

Chain FORWARD (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 332 packets, 28460 bytes)
num   pkts bytes target     prot opt in     out     source               destination
```

Active iptables ruleset confirmed — 7 rules across INPUT chain, FORWARD and OUTPUT as expected

Rules are active in memory but do not survive a reboot by default. iptables-persistent saves the current ruleset to /etc/iptables/rules.v4 and restores it automatically on every boot via a systemd service:

```
shcherba@Ubuntu-26-04:/usr$ sudo apt install iptables-persistent
iptables-persistent is already the newest version (1.0.24).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 3
shcherba@Ubuntu-26-04:/usr$ sudo netfilter-persistent save
run-parts: executing /usr/share/netfilter-persistent/plugins.d/15-ip4tables save
run-parts: executing /usr/share/netfilter-persistent/plugins.d/25-ip6tables save
shcherba@Ubuntu-26-04:/usr$
```

netfilter-persistent saves both IPv4 and IPv6 rulesets for persistent enforcement

5. Deploying the Wazuh Agent on the Windows Endpoint

With the Manager stack confirmed healthy, the Windows 11 VM is enrolled as a monitored endpoint. The agent is installed and configured entirely from PowerShell running as Administrator.

5.1 Download the Agent MSI

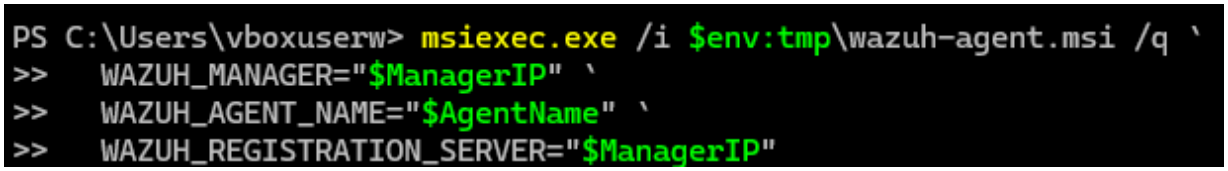
```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.0-1.msi `
  -OutFile $env:tmp\wazuh-agent.msi
```

- Invoke-WebRequest — PowerShell's HTTP client cmdlet. Downloads the MSI directly from Wazuh's official package repository into the Windows temp folder ($env:tmp).

Downloading the Wazuh agent MSI from packages.wazuh.com

5.2 Install with Enrollment Parameters

- msiexec /i /q — installs the MSI package silently without a GUI wizard.
- WAZUH_REGISTRATION_SERVER (port 1515) — contacted once during enrollment to issue the agent a unique cryptographic authentication key.
- WAZUH_MANAGER (port 1514) — the destination for all ongoing encrypted log shipping after enrollment.
- WAZUH_AGENT_NAME — the label shown in the Dashboard agent list. A descriptive name (e.g., win11-soc-lab-endpoint) reads as deliberate and is searchable in alerts.



Silent MSI installation with Manager IP and agent name passed as parameters

> **Enrollment vs. Data Channel**
> Wazuh deliberately separates enrollment (port 1515, one-time) from log shipping (port 1514, continuous). A compromised endpoint cannot use the data channel to replay an enrollment or impersonate a new agent without completing a legitimate one-time key exchange first.

5.3 Start the Agent Service

```
NET START WazuhSvc
```

The installer configures the service for Automatic startup by default. It will start on every subsequent boot without further action. Verify via services.msc: the WazuhSvc service should show Startup Type: Automatic.

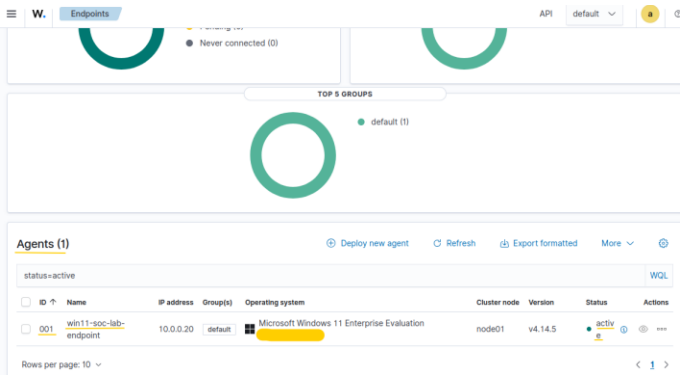
5.4 Verify Enrollment — CLI and Dashboard

Lists all enrolled agents with their connection status. The Windows endpoint should appear as Active.

```
shcherba@Ubuntu-26-04:~/wazuh-install$ sudo /var/ossec/bin/agent_control -l

Wazuh agent_control. List of available agents:
   ID: 000, Name: Ubuntu-26-04 (server), IP: 127.0.0.1, Active/Local
   ID: 001, Name: win11-soc-lab-endpoint, IP: any, Active
```

agent_control -l: Agent 001 (win11-soc-lab-endpoint) confirmed Active alongside Agent 000 (server self-monitoring)



Dashboard → Endpoints: Windows 11 Enterprise agent Active, v4.14.5, on node-01

> **Agentless Monitoring**
> For devices where an agent cannot be installed (physical routers, older appliances, network gear), Wazuh provides an agentless monitoring module where the device pushes logs outward via syslog. No agent process runs on the device itself.

6. Writing a Custom Correlation Rule

Before creating the custom rule, one additional configuration is required: enabling raw event archiving. This is essential for testing rule logic against real event data.

```
# In /var/ossec/etc/ossec.conf, inside the <global> block:
# <logall>yes</logall>
sudo systemctl restart wazuh-manager
```

With logall enabled, Wazuh writes every incoming raw event string to /var/ossec/logs/archives/archives.log regardless of whether it matched a detection rule — the complete, unfiltered stream of telemetry for rule development and testing.

6.1 Identify the Base Rule

Windows failed-logon events (Event ID 4625) are already decoded and alerted by Wazuh's built-in ruleset as rule 60122 ("Logon Failure — Unknown user or bad password," level 5). Confirm this rule is firing on the live endpoint before building a correlation on top of it:

```
sudo tail -n 10 /var/ossec/logs/alerts/alerts.json \
  | jq '.rule.id, .rule.description'
```

```
shcherba@Ubuntu-26-04:~/wazuh-install$ sudo tail -n 10 /var/ossec/logs/alerts/alerts.json | jq '.rule.id, .rule.description'
"60122"
"Logon Failure - Unknown user or bad password"
"60122"
"Logon Failure - Unknown user or bad password"
```

Rule 60122 confirmed firing on the Windows endpoint — base rule for the correlation chain

> **Note: jq**
> jq is a lightweight command-line JSON processor. Piping alerts.json through jq . pretty-prints the JSON so individual fields (rule.id, rule.level, agent.name) are actually readable instead of one dense line per event.

6.2 Write the Custom Rule File

A new file is created rather than editing local_rules.xml directly. Wazuh's rule engine scans the entire /var/ossec/etc/rules/ directory automatically — no additional registration in ossec.conf is required.

```
shcherba@Ubuntu-26-04: ~/wazuh-install — sudo nano /var/ossec/etc/rules/brute-force.xml
~/wazuh-install

  GNU nano 8.7.1                          /var/ossec/etc/rules/brute-force.xml *
<group name="local,windows,authentication,brute_force,">

<rule id="100050" level="10" frequency ="5" timeframe="60">

<if_matched_sid>60122</if_matched_sid>

<same_field>win.eventdata.ipAddress</same_field>

<description>
Possible brute force attack:
5+ failed logons from the same source IP
within 60 seconds
</description>

<mitre>
<id>T1110</id>
</mitre>

<options>no_full_log</options>

</rule>

</group>
```

Custom correlation rule 100050 in nano — brute-force.xml

Line-by-line explanation:

Element	Explanation
<group name="local,windows,authentication,brute_force,">	Wraps all rules in the file. Every enclosed rule inherits these four tags for Dashboard filtering, compliance mapping, and group-based correlation. The trailing comma after each tag name is required syntax.
id="100050"	Custom rule ID in the 100000–120000 range reserved for user-defined rules, preventing collision with Wazuh's official ruleset.

Element	Explanation
level="10"	Severity assigned when this rule fires — High band on the severity table. Indicates a strong indicator of compromise.
frequency="5" timeframe="60"	The correlation threshold: the watched condition must be observed 5 times within a 60-second rolling window before this rule fires.
<if_matched_sid>60122</if_matched_sid>	The condition being counted. Watches for rule 60122 (failed logon) firing repeatedly — not the raw logs. Chain: Event 4625 → rule 60122 fires → rule 100050 counts it.
<same_field>win.eventdata.ipAddress</same_field>	Requires all 5 counted firings to share the same source IP address. Without this, 5 random logon failures from different sources would trigger the rule — not a brute-force signature.
<description>	Human-readable text displayed in the Dashboard alert. No effect on detection logic.
<mitre><id>T1110</id></mitre>	Declarative: attaches the MITRE ATT&CK technique ID for Brute Force. This is what populates the Dashboard's MITRE module — no automatic inference occurs.
<options>no_full_log</options>	Suppresses the raw log text from being stored in each alert entry in alerts.json, keeping storage lean once the rule's logic is trusted.

6.3 Set Correct File Ownership and Permissions

```
shcherba@Ubuntu-26-04:~/wazuh-install$ sudo chown root:wazuh /var/ossec/etc/rules/brute-force-rdp.xml
shcherba@Ubuntu-26-04:~/wazuh-install$ sudo chmod 640 /var/ossec/etc/rules/brute-force-rdp.xml
```

File ownership set to root:wazuh; permissions 640 — least-privilege access for the Wazuh service

chmod 640 enforces the principle of least privilege: the owner (root) has full read-write access, the wazuh group can read the rule to load it, and all other users have no access. The wazuh service cannot modify its own rule files.

6.4 Restart the Manager

```
shcherba@Ubuntu-26-04:~/wazuh-install$ sudo systemctl restart wazuh-manager
shcherba@Ubuntu-26-04:~/wazuh-install$ sudo systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
     Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-06-22 01:27:09 +03; 9s ago
 Invocation: 7770339891b9400ba46eb42288cf535b
    Process: 15609 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
      Tasks: 169 (limit: 7477)
     Memory: 1.1G (peak: 1.1G)
        CPU: 16.078s
```

wazuh-manager restarted clean — active (running)

> **Pre-Restart Validation**
> Before restarting the Manager in any production environment, always validate the rule file syntax first using wazuh-logtest. A single malformed XML tag will prevent the entire manager service from starting.

7. Attack Simulation & Detection Validation

With the custom detection rule deployed, the final step is validating it against real attack telemetry. This confirms the entire pipeline end-to-end: Windows endpoint generates events → Agent ships them → Manager decodes and correlates → Alert appears in Dashboard.

7.1 Testing Methodology

External brute-force tools (Hydra, Ncrack) encountered protocol-level restrictions during testing: Windows 11 enforces Network Level Authentication (NLA) on RDP, and local accounts are blocked from accessing administrative shares (C$) across the network boundary by default.

To isolate the SIEM rule logic and validate the detection pipeline without introducing external tool compatibility as a variable, a deterministic Internal Loopback Simulation was used instead. The simulation generates the same Windows Security event artifacts (Event ID 4625, authentication failures) that any real brute-force would produce, making the telemetry and detection validation fully authentic.

7.2 PowerShell Brute-Force Simulation Script

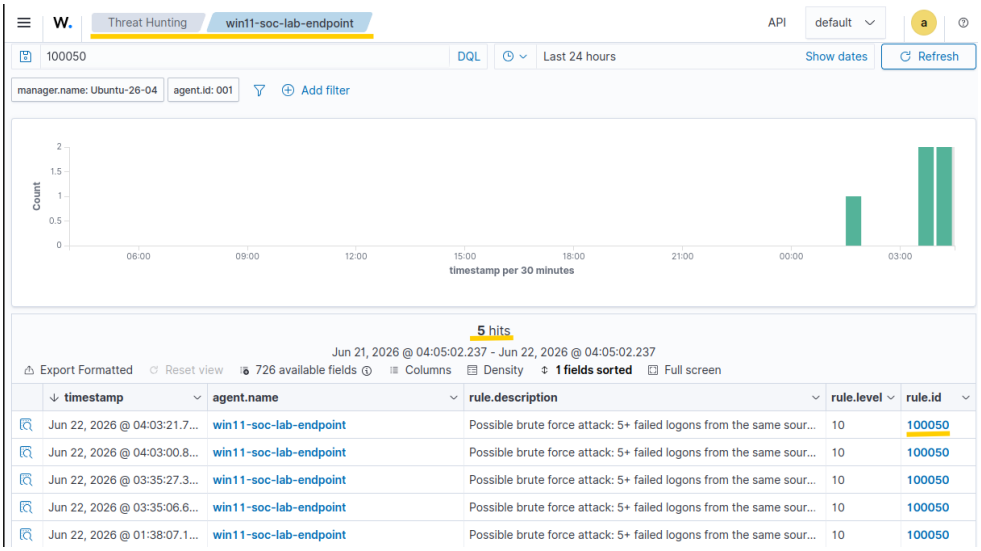
```
PS C:\WINDOWS\system32> for ($i=1; $i -le 10; $i++) {
>>     net use \\10.0.0.20\c$ /user:testuser "WrongPassword$i" > $null 2>&1
>> }
```

PowerShell loop executing 10 sequential failed authentication attempts against the internal network interface

Script Element	Function
for ($i=1; $i -le 10; $i++)	Control loop: 10 iterations executed back-to-back, simulating a burst of rapid authentication attempts.
net use \\\\10.0.0.20\\c$	Network payload: authenticates against the host's own internal network interface IP (not 127.0.0.1) to force traffic through the network stack and generate proper Security log events.
/user:testuser	Targets the designated low-privilege test account created specifically for attack simulation.
"WrongPassword$i"	Generates a unique incorrect credential per iteration (WrongPassword1…WrongPassword10), simulating a credential-stuffing or brute-force footprint.
> $null 2>&1	Redirects both stdout and stderr to null — silently swallows the 10 expected "System error 86" messages to keep the analyst console clean.

7.3 Verify — Dashboard (Threat Hunting)

In the Dashboard, navigate to Threat Hunting, select the win11-soc-lab-endpoint agent, and search for rule ID 100050:



Custom rule 100050 firing in the Threat Hunting module — 5 alerts, level 10, agent win11-soc-lab-endpoint

> **Dashboard Confirms**
>
> Rule 100050 fired correctly: 5 hits visible, each at rule.level: 10, attributed to the correct agent, with timestamps clustering in the expected time range matching the simulation run (I was having a little fun before, so… amount of alerts is slightly bigger, than it should be).

7.4 Verify — CLI (alerts.json)

```
shcherba@Ubuntu-26-04:~/wazuh-install$ sudo grep "100050" /var/ossec/logs/alerts/alerts.json | jq
```

CLI query filtering alerts.json for rule 100050


```
{
  "timestamp": "2026-06-22T04:03:21.760+0300",
  "rule": {
    "level": 10,
    "description": " Possible brute force attack:\n5+ failed logons from the same source IP\nwithin 60 seconds\n",
    "id": "100050",
    "mitre": {
      "id": [
        "T1110"
      ],
      "tactic": [
        "Credential Access"
      ],
      "technique": [
        "Brute Force"
```


Raw alert JSON: rule.level 10, rule.id 100050, MITRE T1110 Brute Force / Credential Access confirmed

> **End-to-End Validation Complete**
>
> The alert JSON confirms: correct rule ID (100050), severity level (10), MITRE technique (T1110), tactic (Credential Access), and technique name (Brute Force). Every layer of the pipeline — telemetry, decoding, correlation, indexing, and presentation — performed as designed.

8. MITRE ATT&CK Mapping

Activity	Technique	Sub-Technique	ID
Credential brute force via SMB loopback simulation	Brute Force	Password Guessing	T1110.001
Custom correlation rule 100050 (detection)	Brute Force	—	T1110
Successful logon following credential compromise	Valid Accounts	Local Accounts	T1078.003
(Future) Audit log clearing to suppress alerts	Indicator Removal	Clear Windows Event Logs	T1070.001
(Future) RDP lateral movement after credential access	Remote Services	Remote Desktop Protocol	T1021.001

9. Conclusion

This lab established a functional home SOC environment mirroring real enterprise SIEM architecture: a dedicated monitoring server (Wazuh Manager + Indexer + Dashboard) receiving and analyzing telemetry from a monitored Windows endpoint, with network-level isolation enforced via a custom iptables ruleset persisted across reboots.

Beyond the infrastructure setup, the lab demonstrated the full detection engineering lifecycle: identifying an existing base rule (60122), understanding the event it monitors (Windows Event ID 4625), designing a correlation rule on top of it (100050) that elevates isolated failures into a pattern with a meaningful alert level, and validating end-to-end that simulated attack traffic flows correctly through every layer of the pipeline and surfaces as expected in both the Dashboard and alerts.json.

Several real obstacles were encountered and documented throughout: disk exhaustion caused by dynamically-allocated VDI snapshot bloat, a broken dpkg state requiring manual package database repair via a stub binary, Ubuntu version compatibility warnings, and the limitations of external brute-force tools against hardened Windows 11 NLA/SMB policies. Each was resolved (almost :)

Key Takeaways

- The Wazuh rule engine separates single-event matching (if_sid, parent–child chains) from temporal correlation (if_matched_sid + frequency/timeframe) — two fundamentally different mechanisms that require different mental models to write correctly.
- same_field is the element that turns a raw event count into a directional brute-force signature. Without it, 5 random failures from different sources would trigger the rule, making it meaningless.
- systemctl status confirms a process is alive. Only an API query against port 9200 confirms the Indexer has actually completed startup and is accepting requests — a critical distinction when troubleshooting silent data-loss scenarios.
- File permissions on custom rule files (root:wazuh, chmod 640) are not cosmetic — incorrect permissions will silently prevent the rule file from loading, with no alert generated.

- Snapshots on dynamically-allocated VDI files accumulate host-side disk usage that is invisible to df -h inside the guest. Under write-heavy workloads (repeated large package installs), this can cause genuine kernel-level write failures that appear contradictory to in-guest space reporting.

Part of an ongoing self-directed Home SOC Lab series. All errors, obstacles, and resolutions documented authentically.

The End.